```
using System.Collections.Concurrent;

WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

var _fruit = new ConcurrentDictionary<string, Fruit>();          ❶

app.MapGet("/fruit", () => _fruit);

app.MapGet("/fruit/{id}", (string id) =>
    _fruit.TryGetValue(id, out var fruit)                        ❷
        ? TypedResults.Ok(fruit)                                 ❸
        : Results.NotFound());                                   ❹


app.MapPost("/fruit/{id}", (string id, Fruit fruit) =>
    _fruit.TryAdd(id, fruit)                                     ❺
        ? TypedResults.Created($"/fruit/{id}", fruit)            ❻
        : Results.BadRequest(new                                 ❼
            { id = "A fruit with this id already exists" }));    ❼

app.MapPut("/fruit/{id}", (string id, Fruit fruit) =>
{

    _fruit[id] = fruit;
    return Results.NoContent();                                  ❽
});

app.MapDelete("/fruit/{id}", (string id) =>
{
    _fruit.TryRemove(id, out _);                                 ❾
    return Results.NoContent();                                  ❾
});

app.Run();
record Fruit(string Name, int stock);
```

❶ Uses a concurrent dictionary to make the API thread-safe

❷ Tries to get the fruit from the dictionary. If the ID exists in the dictionary, this returns true . . .

❸ . . . and we return a 200 OK response, serializing the fruit in the body as JSON.

❹ If the ID doesn’t exist, returns a 404 Not Found response

❺ Tries to add the fruit to the dictionary. If the ID hasn’t been added yet. this returns true . . .

❻ . . . and we return a 201 response with a JSON body and set the Location header to the given path.

❼ If the ID already exists, returns a 400 Bad Request response with an error message